Cartucho ZCard com dois W25Q128 e USB MSC

Correção importante: FAT16, não FAT32

Para uma mídia de **32 MiB**, use FAT16. FAT32 é voltado para volumes maiores e não é a escolha natural para este tamanho. A imagem abaixo usa:

```
Plain Text
```

```
capacidade total: 32 MiB
setores:          65.536
setor:            512 bytes
cluster:          2 setores = 1 KiB
filesystem:      FAT16
```

O Windows monta uma imagem FAT16 desse tamanho normalmente.

Divisão física entre os dois chips

Cada W25Q128 tem:

```
Plain Text
```

```
16 MiB = 16.777.216 bytes = 32.768 setores de 512 bytes
```

A mídia lógica de 32 MiB fica:

LBA lógico	Endereço lógico	Chip	LBA local	Endereço local
0–32767	`0x000000–0xFFFFFF`	W25Q128-A	0–32767	`0x000000–0xFFFFFF`
32768–65535	`0x1000000–0x1FFFFFF`	W25Q128-B	0–32767	`0x000000–0xFFFFFF`

Regra do firmware:

```
Plain Text
```

```
uint32_t address = lba * 512u + offset;
uint8_t chip = address >= 0x01000000u ? 1 : 0;
```

```
uint32_t local = address & 0x00FFFFFFu;
```

A leitura deve desativar o `CS#` do primeiro chip antes de ativar o `CS#` do segundo.

Layout FAT16 exato

A imagem usa 65.536 setores, numerados de 0 a 65.535.

Região	LBA inicial	Quantidade de setores	LBA final
Boot sector	0	1	0
FAT 1	1	128	128
FAT 2	129	128	256
Root directory	257	32	288
Área de dados	289	65.247	65.535

Parâmetros FAT16:

Plain Text

```
bytes_per_sector       = 512
sectors_per_cluster    = 2
reserved_sectors       = 1
number_of_FATs         = 2
root_entries           = 512
sectors_per_FAT        = 128
media                   = 0xF8
volume_sectors          = 65536
first_data_sector       = 289
```

Os primeiros clusters ficam assim:

Plain Text

```
cluster 2 → LBA 289–290
cluster 3 → LBA 291–292
cluster 4 → LBA 293–294
```

Arquivo GAME.ZCARD

O arquivo `GAME.ZCARD` é armazenado na área de dados da FAT. Para manter o primeiro teste simples, o gerador coloca o arquivo a partir do cluster 2.

O arquivo pode ocupar até aproximadamente:

Plain Text

```
área de dados total ≈ 31,9 MiB
```

Na prática, deixe margem para:

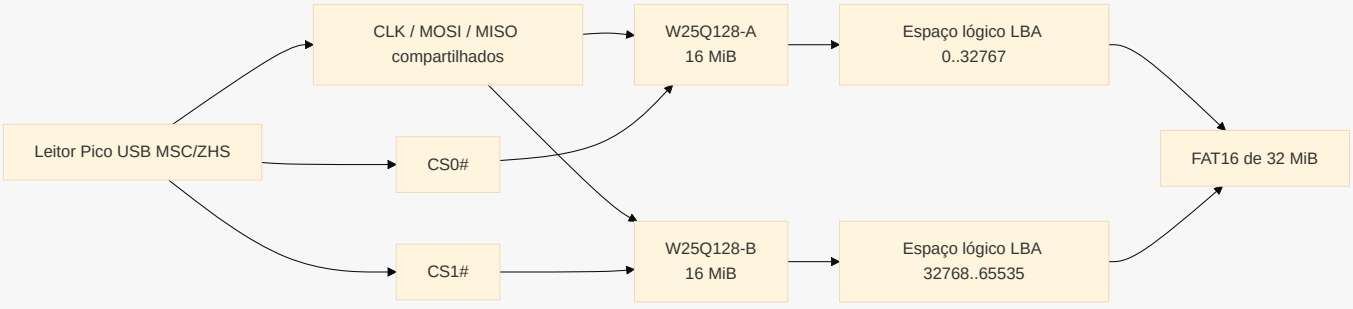
Plain Text

```
GAME.ZCARD
README.TXT
META.TXT
```

O `GAME.ZCARD` não deve ser gravado diretamente no início da flash, porque os primeiros setores são o boot sector, FAT e diretório raiz.

Esquema lógico do cartucho

mermaid



Esquema elétrico

Barramento compartilhado

Plain Text

```
                                ┌──────────────────┐
Pico 3V3 ──────────────────────┤ VCC  W25Q128-A   │
Pico GND ──────────────────────┤ GND              │
Pico GP18/SCK ─────────────────┤ CLK              │
Pico GP19/MOSI ────────────────┤ DI               │
```

```
Pico GP16/MISO ◄──────────┤ DO              │
Pico GP17/CS0# ───────────┤ CS#             │
                          └─────────────────┘


                          ┌─────────────────┐
Pico 3V3 ─────────────────┤ VCC  W25Q128-B  │
Pico GND ─────────────────┤ GND             │
Pico GP18/SCK ────────────┤ CLK             │
Pico GP19/MOSI ───────────┤ DI              │
Pico GP16/MISO ◄──────────┤ DO              │
Pico GP20/CS1# ───────────┤ CS#             │
                          └─────────────────┘
```

Componentes por chip

Plain Text

```
VCC ── 100 nF ── GND       (um capacitor junto de cada CI)
WP# ── 10 kΩ ── 3V3
HOLD#/RESET# ── 10 kΩ ── 3V3
```

Adicione também no conector de alimentação do cartucho:

Plain Text

```
3V3 ── 1 µF a 4,7 µF ── GND
```

Pinagem SOIC-8 típica

Plain Text

```
                    ┌──────────────┐
CS#           1 ──┤              ├── 8 VCC
DO / MISO     2 ──┤  W25Q128JV   ├── 7 HOLD#/RESET#
WP#           3 ──┤              ├── 6 CLK
GND           4 ──┤              ├── 5 DI / MOSI
                    └──────────────┘
```

Confirme a marcação e o datasheet do lote antes de fabricar a PCB.

Regra de seleção do chip

Plain Text

```
bool read_logical(uint32_t address, uint8_t *dst, size_t count) {
    while (count) {
        uint8_t chip = address < 0x01000000u ? 0 : 1;
        uint32_t local = address & 0x00FFFFFFu;
        size_t until_boundary = 0x01000000u - local;
        size_t part = count < until_boundary ? count : until_boundary;

        flash_select(chip);
        flash_read_local(local, dst, part);
        flash_deselect(chip);

        address += part;
        dst += part;
        count -= part;
    }
    return true;
}
```

Nunca deixe os dois `CS#` baixos ao mesmo tempo. O chip não selecionado deve permanecer com `CS#` alto para não dirigir o MISO.

MSC `READ10`

O callback recebe um LBA e um deslocamento dentro do setor:

Plain Text

```
uint32_t logical = lba * 512u + offset;
read_logical(logical, buffer, bufsize);
```

Para W25Q128-A/B:

Plain Text

```
logical < 0x01000000 → CS0#, local = logical
logical >= 0x01000000 → CS1#, local = logical - 0x01000000
```

A mídia MSC anuncia:

Plain Text

```
block_count = 65536
block_size  = 512
writable    = false
```

Criar a imagem no Windows

O pacote inclui o script `make_zcard_fat16.py`. Ele recebe um arquivo `.zcard`, cria `zcard_32mib.img` e divide a imagem em:

```
Plain Text

chip_a_16mib.bin
chip_b_16mib.bin
```

Os arquivos binários podem ser gravados nos chips separadamente:

```
Plain Text

chip_a_16mib.bin → W25Q128-A
chip_b_16mib.bin → W25Q128-B
```

Depois de instaladas na PCB, o leitor deve enxergar a imagem como uma única mídia.

Gravação

A gravação pode ser feita com:

- gravador SPI de 3,3 V;
- adaptador SOIC-8;
- um firmware de gravação no Pico, em etapa posterior.

Para cada chip:

```
Plain Text

auto-detect/JEDEC
apagar
gravar o BIN correspondente
ler novamente
comparar SHA-256
```

Não grave `zcard_32mib.img` inteiro em cada chip. Grave as duas metades separadamente.

Limitações

- O Windows pode tentar escrever metadados mesmo em uma unidade marcada como somente leitura; o callback `WRITE10` deve sempre falhar.

- Não use MSC e ZHS acessando a flash ao mesmo tempo.
- O arquivo `.ZCARD` precisa caber na área de dados FAT16, não na capacidade bruta sem estrutura.
- Atualizações exigem uma política própria; não sobrescreva a ROM MASTER sem transação e verificação.
- O primeiro protótipo deve ser somente leitura.